R, Python, and Quarto

A primer for IR professionals with diverse familiarity with these languages

Auburn University Office of Institutional Research

October 2026

Table of contents

1. What you are looking at 2
2. YAML: the header 2
   2.1. YAML beyond one document: `_quarto.yml` 3
3. Markdown: the text 3
4. How to say it 4
5. R in twelve minutes 5
   5.1. Assignment, vectors, and types 5
   5.2. Data frames 5
   5.3. The pipe and the five verbs 6
   5.4. If you would rather just write SQL 6
   5.5. Coming from Access, by way of Oracle 7
   5.6. Functions and packages 8
   5.7. Help 8
6. R and Python, side by side 9
   6.1. R 9
   6.2. Python 10
7. Code chunks in Quarto, properly 10
   7.1. The fence 10
   7.2. The options you will actually use 10
   7.3. The options in practice 10
   7.4. Inline code 11
   7.5. Parameters 11
   7.6. Execution order and the fresh session 12
   7.7. Other engines in the same file 12
8. Markdown and Quarto features worth knowing 12
9. The shape of your data, without the data 12
   9.1. Databases 12
   9.2. Access 13
   9.3. Files and folders 13
   9.4. The trick behind the tricks 14
10. Where am I? Working directories and paths 14
   10.1. The Terminal is a different place 14
11. Useful code to keep 15
   11.1. Read anything 15
   11.2. Paths that work everywhere 15
   11.3. A table that goes anywhere 15
   11.4. A chart that goes anywhere 15
   11.5. Render many 16
   11.6. Lock the versions 16
   11.7. Secrets 16
   11.8. A phrasebook of everyday R 16

12. Git, briefly 17
13. Errors you will see on the first day 18
14. Words to use correctly 18
15. Two things that exist, and we did not cover 19
16. Where to go next 19

Every R chunk in this document actually ran when it was rendered. The output under each one is real. That is the point of Quarto, and it is also why this document cannot lie to you about what the code does.

1. What you are looking at

A Quarto document is a plain-text file, `.qmd`, with three kinds of content:

1. A **YAML header** at the top, between two `---` lines: title, author, output format, parameters, options.
2. **Markdown** text: headings, paragraphs, lists, links, tables.
3. **Code chunks**: fenced blocks that run when you render. R, Python, SQL, and others. Their output (tables, charts, numbers) is placed into the document where the chunk sits.

Rendering means: run every chunk from top to bottom in a fresh session, weave the output into the text, hand the result to Pandoc, and let Pandoc produce HTML, PDF, Word, slides, or whatever the header asked for. Same file, any container.

The older name for this idea is **literate programming** (Knuth, 1984): write the explanation and the computation together, so the document is the program and the program is the document. R Markdown did it for R starting in 2012; Quarto, released by Posit in 2022, does it for R, Python, Julia and Observable with one tool, and adds websites, books, dashboards, and manuscripts.

2. YAML: the header

YAML (say "yammel") is the settings language at the top of every Quarto file. It is indentation-sensitive: two spaces per level, never tabs, and a colon-space between a key and its value. That is almost all there is to it.

```yaml
---
title: "Program Performance Preview"
subtitle: "One institution"
author: "Office of Institutional Research"
date: today                    # or "2026-10-20"
date-format: "MMMM D, YYYY"
format:
  html:
    toc: true                  # table of contents in the sidebar
    toc-depth: 2
    embed-resources: true      # images and styles inside the one file
    theme: cosmo               # any Bootswatch theme name
  typst:
    papersize: us-letter
    fontsize: 10pt
params:
  unitid: 100858
execute:
  echo: false                  # defaults for every chunk
  warning: false
  message: false
---
```

Things to know:

- **`format:` can be one word or a tree.** `format: html` is enough. `format:` with `html:` and options under it is the same thing with settings. Several formats listed means Render can produce any of them.

- **Quoting.** Strings with colons or special characters need quotes. `title: "Results: 2025"` needs them; `title: Results` does not. When in doubt, quote.
- **`execute:` sets chunk defaults.** Whatever you put here applies to every chunk unless a chunk overrides it with its own `#|` line. A report puts `echo: false` here once; a teaching document leaves it out.
- **`params:` is the dictionary of things that change between renders.** Each key becomes `params$key` in code.
- **Comments** start with `#`, like R.
- **A wrong indent is the most common YAML error.** The message is usually "YAML parse exception" or "mapping values are not allowed here" with a line number. Look at that line's indentation.

2.1. YAML beyond one document: `_quarto.yml`

A folder with a `_quarto.yml` in it is a **Quarto project**: a website, a book, a set of reports that share settings. The file names the thing and holds settings every document inherits.

```yaml
project:
  type: website
  output-dir: _site

website:
  title: "Office of Institutional Research"
  navbar:
    left:
      - href: index.qmd
        text: Home
      - href: reports.qmd
        text: Reports
      - about.qmd

format:
  html:
    theme: flatly
    toc: true
```

With that file in place, `quarto render` in the Terminal builds the whole site into `_site/`, and `quarto preview` opens it live in a browser while you edit. A fact book, a documentation site, a set of department profiles with an index page: all this one file plus a `.qmd` per page. `type: book` gives chapters and a single PDF; `type: default` is just shared settings.

3. Markdown: the text

Markdown is plain text with a few marks that become formatting. You already know most of it from Slack, Teams, and GitHub.

Write	Get
`# Heading ## Subheading ### Smaller`	headings; `#` count is the level
`**bold** *italic* `code``	**bold** *italic* `code`
`- item` or `1. item`	bullet or numbered list; indent two spaces to nest
`[text](https://url)`	a link
`![caption](image.png)`	an image
`> quoted text`	a block quote
`---` on its own line	a horizontal rule
blank line	a new paragraph; a single line break does nothing
`|a|b|` with `|---|---|` under it	a table (the visual editor is easier)

Write	Get
`::: {.callout-note}` … `:::`	a highlighted box

The visual editor (button at the top left of the editor pane) shows this rendered and writes the marks for you. Switch back to Source to see what it wrote. Learning the marks is a day; after that, plain text is faster than any word processor and it diffs cleanly in git.

4. How to say it

Pronunciation is notable only because getting it wrong marks you as new, and getting it right costs nothing.

Written	Said	Note
R	“are”	
RStudio, Posit	“R studio”, “POZ-it”	Posit is the company; RStudio is the IDE
Quarto	“KWAR-toh”	
YAML	“YAM-ul”	“YAML Ain’t Markup Language”
Markdown	as written	
Pandoc	“PAN-dock”	
Typst	“TIPST”, one syllable	
LaTeX	“LAH-tek” or “LAY-tek”	both are accepted; the X is a Greek chi, so never “lay-tex” like the glove. TeX alone is “tek”
knitr	“NIT-er”	the R chunk engine
dplyr	“dee-PLY-er”	“data pliers”
tidyr	“TIDY-er”	
ggplot2	“gee-gee-plot two”	gg = grammar of graphics
tibble	“TIB-ul”	
purrr	“purr”	the map functions
stringr, readr, lubridate	“STRING-er”, “READ-er”, “LOO-bri-date”	
gt	“gee-tee”	“grammar of tables”
renv	“R-env”	
reticulate	“reh-TIC-you-late”	the Python bridge
CRAN	“kran”	the package archive
`<-`	“gets”	`x <- 5`: “x gets five”
`|>` and `%>%`	“then” or “pipe”	
`::`	“from”	`dplyr::filter`: “filter from dplyr”
`%in%`	“in”	
`NA`	“en-ay”	not “nah”
`.qmd`	“Q-M-D”	
regex	“REJ-ex” or “REG-ex”	both are used

Written	Said	Note
SQL	“sequel” or “S-Q-L”	both are used; Oracle people say “sequel”
JSON	“JAY-son”	
IPEDS, CIP	“EYE-peds”, “sip” or “C-I-P”	you knew these

5. R in twelve minutes

R is a language for data. It was built by statisticians, which shows in three ways: the basic unit is a **vector** (a column), not a single value; missing values are a first-class idea called `NA`; and almost everything you want to do to a column is already a function.

5.1. Assignment, vectors, and types

```
x <- c(12, 15, 9, NA, 22)     # a numeric vector; NA is "missing"
x
```

```
[1] 12 15  9 NA 22
```

```
length(x)
```

```
[1] 5
```

```
mean(x)                       # NA is contagious ...
```

```
[1] NA
```

```
mean(x, na.rm = TRUE)         # ... unless you say otherwise
```

```
[1] 14.5
```

```
x * 2                         # arithmetic is vectorized: no loop needed
```

```
[1] 24 30 18 NA 44
```

```
x > 10                        # so are comparisons; the result is logical
```

```
[1]  TRUE  TRUE FALSE    NA  TRUE
```

`<-` is assignment. `=` works too, but `<-` is what you will see everywhere. Type `Alt -` in RStudio to get it.

The types you meet daily: `numeric`, `character` (text, in quotes), `logical` (`TRUE` / `FALSE`), `factor` (a categorical variable with fixed levels), and `Date`.

5.2. Data frames

A data frame is a table: columns are vectors, all the same length. Nearly everything in IR work is a data frame.

Where things come from, since the names blur together. Base R, what you get with no packages loaded, has the `data.frame`. The **tidyverse** is a family of packages that share a style and are installed and loaded together with `library(tidyverse)`: `dplyr` (verbs on tables), `tidyr` (reshape), `ggplot2` (charts), `readr` (read files), `stringr` (text), `purrr` (loops as functions), `forcats` (factors), `lubridate` (dates), and `tibble`. A **tibble** is the tidyverse's data frame: the same thing underneath, with better manners. It prints ten rows and the column types instead of flooding the console, never turns text into factors behind your back, and never guesses. Anything `readr` or `dplyr` hands you is a tibble; anything base R hands you is a plain data frame. Every function in this document accepts either, so the distinction only matters when you notice one printing differently from the other.

```
library(dplyr)
programs <- readRDS(here::here("data", "ppd_programs.rds"))
dim(programs)                # rows, columns
```

```
[1] 209321     83
```

```
programs$credlev[1:3]        # one column, first three values
```

```
[1] "Bachelor" "Bachelor" "Bachelor"
```

```
tested <- programs |> filter(unitid == 100858, earnings_test != "Not tested")
nrow(tested)
```

```
[1] 76
```

`$` pulls out a column. `df[row, col]` indexes with numbers or names, and counting starts at **1**, not 0.

5.3. The pipe and the five verbs

The pipe `|>` means "then". `x |> f(y)` is `f(x, y)`. It lets you write a data step as a sentence read top to bottom.

The pipe is not tidyverse. This is the twist. `|>` is part of base R since version 4.1 (2021); it works with no package loaded, on any function. The tidyverse popularized the idea years earlier with `%>%` from the `magrittr` package, and `%>%` is what most code on the internet still shows. The two are interchangeable for everything in this workshop. Use `|>`: it is built in, it is what RStudio inserts with `Ctrl+Shift+M` once you tick the option under Tools, Global Options, Code, and it is where new code is going. The five verbs below are `dplyr`, which is tidyverse. The pipe that joins them is R.

Five `dplyr` verbs do most of the work:

Verb	Does	SQL cousin
`filter()`	keep rows that match	`WHERE`
`select()`	keep columns	`SELECT`
`mutate()`	add or change columns	computed columns
`summarise()`	collapse to one row per group	aggregates
`arrange()`	sort	`ORDER BY`

With `group_by()` to set the grouping before `summarise()`, and `left_join()` to join. If you know SQL, you already know dplyr; the words are different and the order is the order you think in.

5.4. If you would rather just write SQL

You can. Many IR people use R as a wrapper around the SQL they already write, and that is a complete way to work. Three forms, same result:

```r
# 1. The query in a string, handed to the database. The common one.
qry <- "SELECT level, COUNT(*) AS n FROM my_schema.population GROUP BY level"
by_level <- DBI::dbGetQuery(con, qry)
```

```
{sql connection=con, output.var="by_level"} -- 2. A SQL chunk: no quotes, syntax highlighting, result lands
in by_level. SELECT level, COUNT(*) AS n FROM my_schema.population GROUP BY level
```

```r
# 3. dplyr that writes the SQL for you and runs it in the database.
by_level <- tbl(con, in_schema("my_schema", "population")) |>
  count(level) |> collect()
```

After any of the three, `by_level` is a data frame and the rest of this document applies: `gt()` to show it, `ggplot()` to chart it, `stopifnot()` to test it. The R you need is about ten lines, and they are all on this page.

5.5. Coming from Access, by way of Oracle

Cecile's notes from moving the IPEDS work out of Access, through Oracle SQL Developer, into Quarto. The same three stops most IR offices make.

Syntax that changes. Each of these will bite once.

Access	Oracle SQL	R / Quarto
`TRANSFORM ... PIVOT` crosstab query	`PIVOT (...)` clause, or `GROUP BY` and pivot afterward	`pivot_wider()`; `pivot_longer()` for the reverse
`IIf(cond, a, b)`	`CASE WHEN cond THEN a ELSE b END`	`if_else(cond, a, b)`; `case_when()` for several
wildcards `*` and `?`	`%` and `_`	`str_detect()` with a pattern; `LIKE` inside your SQL
strings in `"double quotes"`	strings in `'single quotes'`	either, but SQL inside an R string uses single quotes so the R string can use double
`[Enter a term]` prompt	`:term` bind variable	`params$term`
`Nz(x, 0)`	`NVL(x, 0)` or `COALESCE`	`coalesce(x, 0)`; `replace_na()`
`Format(d, "yyyy")`	`TO_CHAR(d, 'YYYY')`	`year(d)` from lubridate
`Left(s, 3)`	`SUBSTR(s, 1, 3)`	`str_sub(s, 1, 3)`

Parameters. In Access a prompt is typed each time and is not reusable. In Oracle a bind variable is reusable logic, but each query asks for its own value. In Quarto a parameter is defined once at the top of the document and every chunk reads it; a file path can be a parameter too, so the same document can run against this year's extract or last year's.

Comments. Access keeps a query's description in a properties box, hidden and small. Oracle has `--` comments in the SQL. Quarto has comments in the code and ordinary prose outside the chunk, which is where the explanation that matters goes: what this question asks, what rule was applied, what to watch for. The comment becomes documentation without being retyped anywhere.

Time. Her measurements, not estimates.

	Access	Oracle	R / Quarto
One IPEDS component	30 minutes to several hours	5 to 15 minutes	up to 5 minutes
How many queries	about 30, run one by one	about 10 condensed, one by one unless in one script	about 10, all run together with the reporting

	Access	Oracle	R / Quarto
Failure mode	stalls on queries returning thousands of rows, and again when copying the results		
What you have at the end	numbers in a spreadsheet, typed into the portal	numbers on screen	the numbers, the queries, the explanation, and the upload file in one rendered document

The hours did not go away because the computer got faster. They went away because thirty separate things became one thing that runs top to bottom and records its own output.

```
programs |>
  filter(stabbr == "AL", exists_in_2024_or_2025 == 1, earnings_test != "Not tested") |>
  group_by(control, credlev) |>
  summarise(programs = n(),
            fail     = sum(earnings_test == "Fail"),
            .groups  = "drop") |>
  mutate(fail_rate = fail / programs) |>
  arrange(desc(fail_rate)) |>
  head(6)
```

```
# A tibble: 6 × 5
  control    credlev               programs  fail fail_rate
  <chr>      <chr>                    <int> <int>     <dbl>
1 For-Profit Undergrad Certificate       20    11   0.55
2 Public     Undergrad Certificate       29     4   0.138
3 Public     Associate                   85     7   0.0824
4 Public     Bachelor                   339     3   0.00885
5 For-Profit Associate                    8     0   0
6 For-Profit Bachelor                    12     0   0
```

Read it aloud: take programs, *then* keep Alabama's tested current ones, *then* group by control and level, *then* count and sum, *then* compute a rate, *then* sort, *then* show six. That is the whole language of data work.

5.6. Functions and packages

A function is `name(arguments)`. Arguments can be given by position or by name, and named is clearer: `mean(x, na.rm = TRUE)`. You write your own with `function`:

```
pct <- function(n, d) if (d == 0) NA else round(100 * n / d, 1)
pct(7, 76)
```

```
[1] 9.2
```

A **package** is a bundle of functions someone published. `install.packages()` once per machine; `library()` once per session, at the top of the file. CRAN, the package repository, holds over 20,000 of them, reviewed and versioned. `dplyr::filter()` means "the `filter` from `dplyr`" and is how you call one function without loading the whole package, or when two packages use the same name.

5.7. Help

`?mean` opens the help page. Every function has one, with examples at the bottom. Error messages name the function; search the message text, with the package name, and the first result is usually the answer.

6. R and Python, side by side

Both are fine. Python is the general-purpose language that also does data; R is the data language that also does general things. For a report-building workflow in an IR office, R has the edge in three places: `ggplot2` for charts, `gt` and friends for tables that go to PDF and Word, and the tidyverse's consistency. Python has the edge in machine learning, in web and API work, and in the size of its hiring pool. Quarto runs both, in the same document if you like, so the choice is not permanent.

	R	Python
Assignment	`x <- 5`	`x = 5`
Comment	`# text`	`# text`
True / false	`TRUE`, `FALSE`	`True`, `False`
Missing	`NA` (typed; `NA_real_`, `NA_character_`)	`None`, `NaN` (pandas uses `NaN`, newer `pd.NA`)
First element	`x[1]`	`x[0]`
Last element	`x[length(x)]`	`x[-1]`
A column	`df$col` or `df[["col"]]`	`df["col"]` or `df.col`
Chaining	`df |> filter(...) |> mutate(...)`	`df.query(...).assign(...)` or method chains
Data frames	`dplyr`, `data.table`	`pandas`, `polars`
Charts	`ggplot2` (grammar of graphics)	`matplotlib`, `seaborn`, `plotnine` (a ggplot port), `plotly`
Packages	CRAN; `install.packages("x")`; `library(x)`	PyPI; `pip install x`; `import x as y`
Function	`f <- function(a, b = 2) { a + b }`	`def f(a, b=2): return a + b`
Loop	`for (i in 1:10) { }` (rarely needed)	`for i in range(10):` (common)
Vectorized math	`x * 2` on any vector	`x * 2` on a NumPy array or Series, not a list
String glue	`glue("Hello {name}")` or `paste0()`	`f"Hello {name}"`
Project isolation	RStudio Project + `renv`	virtual environment + `requirements.txt`
Notebook	`.qmd` (or `.Rmd`)	`.ipynb`; Quarto renders those too
Database	`DBI` + `odbc`, `dbplyr` writes the SQL	`sqlalchemy`, `pyodbc`, pandas `read_sql`

Three habits that trip people crossing over:

- **Indexing starts at 1 in R, 0 in Python.** `x[1]` is the first thing in R and the second in Python.
- **R is vector-first.** Where a Python person reaches for a loop, an R person applies the operation to the whole column. `ifelse(x > 10, "big", "small")` does a thousand rows at once.
- **R's `NA` is strict.** `NA == 1` is `NA`, not `FALSE`. Use `is.na(x)`. Sums and means return `NA` unless you say `na.rm = TRUE`. This is a feature; silent zeros are how IR numbers go wrong.

The same job in both languages, for the record:

6.1. R

```
tested |>
  filter(credlev == "Bachelor") |>
  group_by(cip2_title) |>
```

```r
  summarise(n = n(), median_earn = median(md_earn_wne_p4, na.rm = TRUE)) |>
  arrange(desc(median_earn))
```

6.2. Python

```python
(tested[tested["credlev"] == "Bachelor"]
   .groupby("cip2_title")
   .agg(n=("cip4", "size"), median_earn=("md_earn_wne_p4", "median"))
   .sort_values("median_earn", ascending=False))
```

Quarto can run the Python version in a `{python}` chunk right under the R one. With the `reticulate` package, an R object `df` is visible in Python as `r.df`, and a Python object `df` is visible in R as `py$df`. One document, both languages, one dataset.

7. Code chunks in Quarto, properly

7.1. The fence

````
```{r}
#| label: enrollment-trend
#| echo: false
#| fig-height: 4
#| fig-cap: "Title IV enrollment by level"

ggplot(...) + ...
```
````

The `{r}` names the engine. Lines beginning `#|` at the top of the chunk are **chunk options**; they govern that chunk only. Anything in `execute:` in the YAML header is the default for every chunk.

7.2. The options you will actually use

Option	Effect
`label: name`	names the chunk; appears in the Render progress and in error messages; must be unique
`echo: false`	hide the code, show the output (reports)
`echo: true`	show the code (teaching, documentation)
`eval: false`	show the code, do not run it
`include: false`	run it, show nothing (setup chunks)
`output: false`	run it, show the code, hide output
`warning: false`, `message: false`	silence package chatter
`results: asis`	treat printed text as Markdown (for building text with code)
`fig-height: 4`, `fig-width: 7`	inches
`fig-cap: "..."`, `tbl-cap: "..."`	captions; with a `label` starting `fig-` or `tbl-`, they become numbered and cross-referenceable with `@fig-name`
`cache: true`	keep the result until the code changes; for slow chunks
`error: true`	keep rendering past an error (show it)

7.3. The options in practice

The three chunks every report has, annotated:

````
```{r}
#| label: setup
#| include: false
# Runs, shows nothing. Loads packages and data for everything below.
# include: false hides the code, the output, AND the package start-up
# chatter, so this is where library() calls live.
library(dplyr)
library(ggplot2)
library(gt)
programs <- readRDS(here::here("data", "ppd_programs.rds"))
```
````

````
```{r}
#| label: tbl-glance
#| tbl-cap: "At a glance"
#| echo: false
# Shows the table, hides the code. The label starting tbl- plus the
# caption make it "Table 1" and referenceable as @tbl-glance.
glance |> gt()
```
````

````
```{r}
#| label: fig-margin
#| fig-cap: "Margin above the benchmark"
#| fig-height: 6
#| warning: false
# Shows the chart, hides the code (echo: false came from the YAML header).
# warning: false here silences a ggplot warning about removed NA rows
# for this chunk only; the header already did it globally, but a chunk
# option is the right place when you want to be explicit.
ggplot(tested, aes(earnings_margin, cip4_title)) + geom_col()
```
````

What each line is really saying:

- `include: false` is “do this quietly”. For setup.
- `echo: false` is “show the result, not the recipe”. For readers.
- `echo: true` is “show the recipe”. For colleagues and for teaching.
- `eval: false` is “show the recipe, do not cook”. For documentation of code you do not want to run, such as a database write.
- `warning: false` and `message: false` are “I know, stop telling me”. R packages announce themselves and ggplot warns about dropped rows; readers do not need either.
- `results: hide` is “run it, show the code, hide the printed output”. `output: false` is the Quarto spelling of the same thing.
- `fig-cap` and `tbl-cap` with `fig-`/`tbl-` labels are “number this and let me refer to it”.

7.4. Inline code

A value inside a sentence: backtick, `r`, a space, `nrow(tested)`, backtick. Written in this paragraph it becomes 76 right here. The number in the prose is the number in the data, every time the document renders. No more “update the paragraph” step.

7.5. Parameters

In the YAML header:

```
params:
  unitid: 100858
```

In code: `params$unitid`. Render with a different value from R:

```
quarto::quarto_render("report.qmd", execute_params = list(unitid = 229027))
```

or from the Terminal: `quarto render report.qmd -P unitid:229027`. A loop over a list of IDs is the entire mechanism behind one-report-per-unit.

7.6. Execution order and the fresh session

Render runs chunks top to bottom in a new R session that has nothing in it. This is deliberate. It means a document that renders is a document that works from scratch, on another machine, next year. The common mistake is testing chunks interactively, where earlier experiments linger in memory, and then being surprised that Render fails. If Render fails, the document was depending on something it never loaded. Fix the document.

Setup goes in the first chunk: `library()` calls, reading data, defining helpers. Later chunks use it.

7.7. Other engines in the same file

```
{sql connection=con} SELECT credlev, COUNT(*) AS n FROM programs GROUP BY credlev
```

With `#| output.var: "by_level"` the result lands in an R data frame called `by_level`. The SQL is in the document, next to the sentence it supports.

`{python}` chunks run through `reticulate` (R documents) or Jupyter (Python documents). `{ojs}` runs Observable JavaScript in the browser, for interactive widgets with no server. `{bash}` runs shell commands.

8. Markdown and Quarto features worth knowing

Divs and spans. `::: {.classname}` … `:::` wraps a block; `[text]{.classname}` wraps inline. They carry styling, and they carry instructions:

- `::: {.callout-note title="About the data"}` makes the colored box at the top of the report. Also `.callout-warning`, `.callout-tip`, `.callout-important`.
- `::: {.content-visible when-format="html"}` includes a block only in that format. `unless-format` is the opposite. This is how the 245-row table exists on the web page and not in the PDF.
- `::: {.panel-tabset}` with `##` headings inside makes tabs (HTML only; in PDF they become sections).
- `::: {.columns}` with `::: {.column width="50%"}` inside makes columns.

Cross-references. Label a figure chunk `fig-trend` and write `@fig-trend` in the text. Quarto numbers it and links it. Same for `tbl-`, `sec-` (on headings: `## Results {#sec-results}`), and `eq-`.

Citations. `[@author2024]` with a `.bib` file in the header; the visual editor can insert from Zotero.

Shortcodes. `{{< pagebreak >}}` forces a page break in PDF and Word. `{{< include file.qmd >}}` pulls another file in. `{{< video url >}}` embeds.

Raw blocks. A fenced block marked `{=typst}` or `{=html}` or `{=latex}` is passed straight through to that format and ignored by the others. The notes lines on the IPEDS page of the handout are twelve lines of raw Typst.

The visual editor. The button at the top left of the editor pane switches between source and a Word-like view. Tables, citations, and images are easier there. Everything it does is saved as plain Markdown underneath, so nothing is lost by switching.

9. The shape of your data, without the data

Every prompt in this workshop says "column names and types, not rows." Here is how to get that from wherever your data lives, in one line each. The output of every one of these is safe to paste into an assistant, with one check: read it first, because a column *named* `ssn` is itself a fact, and a DDL export can carry default values or comments with examples in them.

9.1. Databases

Where	What to run	Gives
Oracle, SQL Developer	`DESCRIBE my_schema.my_table`	columns, types, nullable

Where	What to run	Gives
Oracle, any client	`SELECT column_name, data_type, data_length, nullable FROM all_tab_columns WHERE owner = 'MY_SCHEMA' AND table_name = 'MY_TABLE' ORDER BY column_id`	the same, as rows you can save
Oracle, comments	`all_tab_comments`, `all_col_comments`	the documentation someone once wrote
Oracle, the whole definition	`SELECT DBMS_METADATA.GET_DDL('TABLE', 'MY_TABLE', 'MY_SCHEMA') FROM dual`	the CREATE statement; same for `'VIEW'`, `'PACKAGE'`
Oracle, what feeds what	`all_dependencies`, `all_constraints`, `all_views` (column `text`)	lineage, keys, view SQL
Snowflake	`DESCRIBE TABLE my_db.my_schema.my_table`; `SHOW COLUMNS IN TABLE ...`; `SELECT GET_DDL('TABLE', '...')`	same ideas, Snowflake spelling
Snowflake, everything at once	`SELECT * FROM my_db.INFORMATION_SCHEMA.COLUMNS WHERE table_schema = 'MY_SCHEMA'`	one row per column for a whole schema
SQL Server	`sp_help 'my_table'`; `INFORMATION_SCHEMA.COLUMNS`	
Any database, from R	`DBI::dbListTables(con)`; `DBI::dbListFields(con, "my_table")`	names only
Any database, from R	`DBI::dbGetQuery(con, "SELECT * FROM my_table WHERE 1 = 0")`	the columns with zero rows; `str()` it
Any database, in RStudio	the **Connections** pane	click to expand schemas, tables, columns

9.2. Access

What to run	Gives
Database Tools, **Database Documenter**, pick objects, uncheck the data options	a report of every table, field, type, and query SQL
In any query, View, **SQL View**	the query's SQL, to copy
A short VBA loop over `CurrentDb.TableDefs` and `QueryDefs` writing names, fields and `.SQL` to a text file	the whole database's logic as plain text. This is what Auburn did before porting. Ask an assistant to write the loop; the prompt contains nothing sensitive.

9.3. Files and folders

Where	What to run	Gives
A CSV	`readLines("file.csv", n = 1)`	the header row only
An Excel workbook	`readxl::excel_sheets("f.xlsx")`; `readxl::read_excel("f.xlsx", n_max = 0)`	sheet names; column names with no rows

Where	What to run	Gives
A SAS or SPSS file	`haven::read_sas("f.sas7bdat", n_max = 0)`	columns and labels, no rows
A folder	`fs::dir_tree("path")`; `list.files("path", recursive = TRUE)`	the structure
A folder, Windows	`dir /s /b > files.txt` in a command prompt; or PowerShell `Get-ChildItem -Recurse | Select Name, Length, LastWriteTime | Export-Csv files.csv`	every file with size and date
A data frame in R	`str(df)`; `dplyr::glimpse(df)`; `sapply(df, class)`	names and types; `glimpse` shows the first values, so look before pasting
A data frame in R, nothing but shape	`shape_of(df)` from `setup/02_shape_of_my_data.R`	names, types, missing, distinct, no values
A data frame in Python	`df.dtypes`; `df.info()`	
A Qualtrics survey	the survey export as QSF, or `qualtRics::survey_questions()`	questions and types, no responses
A dashboard	Qlik data model viewer; Power BI model view; Tableau data source tab	the fields it uses

9.4. The trick behind the tricks

You can ask the assistant to write the metadata query. "Write an Oracle query that lists every column, type and comment for the tables in schema X" contains nothing but a schema name. Run what it gives you, read the result, paste the result. The assistant then knows your data's shape as well as you do, and has never seen a row.

Keep the result. A text file with table, column, type, and one line of meaning is the data dictionary most offices never get around to writing, and once it exists every future prompt starts by pasting it.

10. Where am I? Working directories and paths

The single most common first-day failure is "cannot open file". The cause is always the same: R is looking in a different folder than you think.

The working directory is the folder R treats as "here". `getwd()` prints it. A relative path like `data/x.rds` is resolved from there.

Three rules keep this from ever being a problem:

1. **Open the project, not the file.** Double-click `IR-Quarto-Workshop.Rproj` (or File > Open Project). RStudio sets the working directory to the project folder and shows the project name at the top right. Opening a lone `.qmd` from Explorer does not do this.
2. **Build paths with `here()`.** `here::here("data", "x.rds")` starts from the project root no matter where the working directory is, including inside a Quarto render, which runs each document from its own folder. This is why the exercises use it.
3. **Never `setwd()` in a file you will share.** It bakes in your machine's folder names. Jenny Bryan's rule: "If the first line of your script is `setwd("C:/Users/jenny/...")`, I will come into your office and set your computer on fire."

For the workshop, put the project in the same place on every machine. Windows: `C:\IR-Quarto-Workshop`. Mac: your home folder, so `~/IR-Quarto-Workshop`. Then every instruction we give is the same for everyone, and `cd` is one short line.

10.1. The Terminal is a different place

RStudio has a **Console** (R) and a **Terminal** (your operating system's shell). The `quarto` command lives in the Terminal. The Terminal has its own working directory, and it starts in the project folder when the project is open. If you ever need to move:

```
cd C:\IR-Quarto-Workshop            # Windows
cd ~/IR-Quarto-Workshop             # Mac
cd exercises                        # down one folder
cd ..                               # up one folder
pwd                                 # where am I (print working directory)
ls                                  # what is here (dir on Windows cmd)
quarto render 02_my_institution.qmd
```

Tab completes names. Up arrow recalls the last command. That is all the shell most people ever need.

11. Useful code to keep

11.1. Read anything

```
readr::read_csv("file.csv")                    # fast, guesses types, tibble
readxl::read_excel("file.xlsx", sheet = 2)
haven::read_sas("file.sas7bdat")               # also read_spss, read_dta
readRDS("file.rds")                            # R's own format; keeps types
DBI::dbGetQuery(con, "SELECT ...")             # from a database
jsonlite::fromJSON("https://api.example/...")  # from an API
```

11.2. Paths that work everywhere

```
here::here("data", "ppd_programs.rds")
```

`here()` finds the project root (the folder with the `.Rproj`) and builds the path from there. It works on Windows, Mac, Linux, Posit Cloud, and inside a Quarto render, whatever the working directory happens to be. Never type `C:/Users/you/...` in a file you will share.

11.3. A table that goes anywhere

```
library(gt)
df |>
  gt(groupname_col = "credlev") |>
  cols_label(cip4_title = "Program", md_earn_wne_p4 = "Median earnings") |>
  fmt_currency(md_earn_wne_p4, decimals = 0) |>
  fmt_number(t4enrl_prg_2025, decimals = 0) |>
  sub_missing(missing_text = "") |>
  tab_header(title = "Tested programs") |>
  tab_style(cell_fill("#F8D7DA"), cells_body(rows = earnings_test == "Fail"))
```

`gt` renders to HTML, PDF (LaTeX and Typst), and Word. One recipe, every output.

11.4. A chart that goes anywhere

```
library(ggplot2)
ggplot(df, aes(x = award_year, y = enrolled, colour = credlev)) +
  geom_line() + geom_point() +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = NULL, y = "Students", colour = NULL, title = "Enrollment") +
  theme_minimal()
```

`ggplot2` is a grammar: data, aesthetics (`aes`: what maps to x, y, color), geoms (lines, points, bars), scales, labels, theme. Every chart is those six things. Change one and the rest stays.

11.5. Render many

```
for (id in c(100858, 229027, 228529)) {
  quarto::quarto_render("report.qmd",
                        execute_params = list(unitid = id),
                        output_file    = paste0("report_", id, ".html"))
}
```

11.6. Lock the versions

```
renv::init()       # once per project: starts tracking packages
renv::snapshot()   # after installing something: writes renv.lock
renv::restore()    # on another machine: installs exactly those versions
```

The lockfile is what makes "the report rendered last year is provably built on the same software" a true sentence.

11.7. Secrets

```
keyring::key_set("my_service", "my_user")        # once, types into a box
pwd <- keyring::key_get("my_service", "my_user")  # at run time
```

Passwords live in the operating system's credential store. They are never in a file, so they are never in git, a zip, an email, or a prompt.

11.8. A phrasebook of everyday R

You want to	Write
look at a data frame	`glimpse(df)`, `head(df)`, `View(df)` (opens a viewer tab)
count things	`df |> count(credlev)`; `count(credlev, sort = TRUE)`
unique values	`distinct(df, credlev)`; `unique(df$credlev)`
rows where	`filter(df, stabbr == "AL", t4enrl_prg_2025 > 100)` (comma means and; `|` means or)
rows where in a list	`filter(df, stabbr %in% c("AL", "GA", "MS"))`
rows with missing	`filter(df, is.na(md_earn_wne_p4))`; not missing: `!is.na(...)`
new column, by rule	`mutate(df, size = if_else(n > 100, "large", "small"))`
new column, several rules	`mutate(df, band = case_when(n < 50 ~ "small", n < 500 ~ "mid", TRUE ~ "large"))`
same change to many columns	`mutate(df, across(starts_with("pct_"), ~ .x / 100))`
rename	`rename(df, enrolled = t4enrl_prg_2025)`
sort descending	`arrange(df, desc(enrolled))`
group and total	`df |> group_by(credlev) |> summarise(n = n(), total = sum(enrolled, na.rm = TRUE))`
percent of total	`mutate(df, share = n / sum(n))` after `summarise`
wide to long	`pivot_longer(df, cols = starts_with("t4enrl_prg_"), names_to = "year", values_to = "n")`

You want to	Write
long to wide	`pivot_wider(df, names_from = credlev, values_from = n)`
join	`left_join(a, b, by = "unitid")`; `by = c("opeid6", "credlev", "cip4")` for a compound key
clean messy column names	`janitor::clean_names(df)` (lower case, underscores)
text contains	`filter(df, str_detect(cip4_title, "Nursing"))`
text to number	`as.numeric(x)`; `parse_number("$1,234")` handles symbols
round, format	`round(x, 1)`; `scales::comma(x)`; `scales::dollar(x)`; `scales::percent(x, 0.1)`
build a sentence	`glue("{n} programs, {fails} fail")`
dates	`lubridate::ymd("2026-10-20")`; `year(date)`; `floor_date(date, "month")`
check an assumption	`stopifnot(nrow(df) == 209321)`; stops the render if false, with a message
say something while rendering	`message("Read ", nrow(df), " rows")`; goes to the Render tab, not the report
a quick table	`table(df$credlev)`; `janitor::tabyl(df, credlev, earnings_test)`
write out	`write_csv(df, "out.csv")`; `writexl::write_xlsx(df, "out.xlsx")`; `saveRDS(df, "out.rds")`
what version of everything	`sessionInfo()`; put it in an appendix chunk

Two habits that pay off: name things as nouns that say what they hold (`tested`, `by_level`, not `df2`), and keep every chunk short enough that you can say in one sentence what it does. If you cannot, split it.

12. Git, briefly

Version control is the part of this toolkit that outlasts every other decision. Quarto may be replaced; git will not be, and the habit transfers to every tool after it.

What it is. A history of every change to every file in a folder, with who, when, and why. Not backups: a backup is a copy; git is the diff. You can see what changed between the report approved in March and the one that went out in October, line by line, and recover either.

Why an IR office wants it. "Where does this number come from?" has always had an answer in a well-run office: a defined, central place. With git, that place also remembers everything. A rule changed in 2024, a column renamed in 2025, a fix in 2026: each is a commit with a message. When someone leaves, their work stays readable. When the SIS changes, the tests written today become the acceptance criteria for tomorrow.

The minimum you need. Three verbs cover ninety percent of it.

Verb	Means	In RStudio
commit	save a named snapshot of the changed files	Git tab, tick files, Commit, write a message
push	send your commits to GitHub (or GitLab, Azure DevOps)	Push button
pull	get commits others sent	Pull button

With `usethis::use_git()` and `usethis::use_github()` a project becomes a repository and gets a home on GitHub in two commands. RStudio's Git tab shows what changed and lets you commit without the command line.

What never goes in. Data with people in it. Passwords, keys, connection strings. Rendered output you can regenerate. The `.gitignore` file lists them so they cannot go in by accident. The workshop project's `.gitignore` is a reasonable starting point for an IR repository: it excludes the raw data, rendered files, and `.Renviron`.

A good commit message says what changed and why, in one line: "Exclude dual-enrollment converts from the freshman cohort (Decision 14)". Future you is the reader.

Public or private? The logic can be public: a repository of SQL, R and Quarto that contains no data is exactly the "readable by people and tools" asset the white paper describes. Many IR offices keep theirs private anyway. Either is fine. What matters is that it exists.

13. Errors you will see on the first day

Message	Cause	Fix
`could not find function "filter"`	package not loaded	`library(dplyr)` in the setup chunk
`object 'df' not found`	chunk above did not run, or a typo	check the chunk order; Render runs from the top
`there is no package called 'gt'`	not installed	`install.packages("gt")` once
`cannot open file 'data/x.rds'`	wrong working directory	use `here::here()`
`non-numeric argument to binary operator`	a number column is text	`as.numeric()`; look for `"privacy suppressed"`
`unexpected symbol` / `unexpected '}'`	a typo in the code: missing comma, quote, or paren	RStudio underlines it in red; find the red
`Error in parse_block` with a line number	a chunk option typo (`#|` lines)	check the `#|` lines; `key: value`, one per line
`Quarto render failed` with nothing else	scroll up in the Render tab; the real error is above	

14. Words to use correctly

Reproducible. The same inputs and the same code produce the same output, on any machine, on any day. A rendered Quarto document with a lockfile is reproducible by construction.

Literate programming. Code and explanation in one document, in the order a human would read. Quarto is a literate programming tool.

Tidy data. One observation per row, one variable per column, one table per kind of thing. The tidyverse verbs assume it; most IR extracts are already close.

Vectorized. An operation applied to a whole column at once. R's default. Faster to run and shorter to read than a loop.

Parameterized report. One template, values supplied at render time. The basis of one-per-department, one-per-college, one-per-institution.

Engine. The thing that runs the chunks: `knitr` for R documents, `jupyter` for Python documents. Quarto picks based on the first chunk.

Pandoc. The document converter under Quarto. Markdown in, fifty formats out. Quarto adds the execution, the project system, and the publishing.

Typst. A modern typesetting system, bundled with Quarto, that makes PDFs without LaTeX. Compiles in a second; LaTeX takes a minute and a two-gigabyte install.

Lockfile. `renv.lock`: the exact version of every package a project used. What lets you say "validated with these versions."

Chunk. One fenced block of code. The unit of execution and of documentation.

Tidyverse. A family of R packages that share one style, loaded together with `library(tidyverse)`: `dplyr`, `tidyr`, `ggplot2`, `readr`, `stringr`, `purrr`, `forcats`, `lubridate`, `tibble`. Not R itself; a set of packages on top of it. Most of this workshop's code is tidyverse.

Tibble. The tidyverse's data frame. Same thing as base R's `data.frame` underneath, with better manners: prints the first ten rows, shows column types, never converts strings to factors. From the `tibble` package, which `dplyr` and `readr` use for everything they return.

Pipe. `|>`. "Then." Base R, since 4.1; not tidyverse. The older `%>%` is tidyverse (`magrittr`) and means the same thing.

Render. Run the chunks, assemble the document, produce the output. "Knit" is the older R Markdown word for the same thing.

Grammar of graphics. The idea behind `ggplot2`: a chart is data mapped to visual properties, drawn with geometric objects, on scales, in a coordinate system. Learn the six parts once and every chart is a combination of them.

15. Two things that exist, and we did not cover

Shiny: interactive dashboards. A Quarto report is fixed once rendered. When the reader needs to pick a term, a college, a cohort, and see the page change, that is Shiny, an R package from Posit for interactive web apps. The same `dplyr` and `ggplot2` code goes inside it; what changes is that the reader's clicks become inputs. It needs a server to run on (Posit Connect, Shiny Server, or shinyapps.io), which is the cost. Two lighter paths: Quarto's `dashboard` format gives a dashboard layout without a server, and **Shinylive** runs small Shiny apps inside a static web page with no server at all. Enrollment dashboards, data-request intake forms, and "what if" tuition models are the usual IR uses. Start at shiny.posit.co.

Posit Cloud: RStudio in a browser. The same RStudio, hosted by Posit, with nothing to install. Useful for teaching and for trying things on a machine you cannot install software on. It is a hosted service on servers your institution does not control, with no FERPA agreement unless your institution has made one, so it is not a place for student records. Everything in this workshop runs on your own laptop for that reason. posit.cloud, free tier available.

16. Where to go next

R for Data Science, 2nd ed., free online	r4ds.hadley.nz
Quarto guide	quarto.org/docs/guide
Quarto gallery, to see what is possible	quarto.org/docs/gallery
ggplot2 book, free online	ggplot2-book.org
gt package site	gt.rstudio.com
Posit cheat sheets, one page each	posit.co/resources/cheatsheets
Python for R users (and the reverse)	rstudio.github.io/reticulate
Tidy data paper (Wickham, 2014)	vita.had.co.nz/papers/tidy-data.html